

Pointers

Engr.Tehseen Ahsan

CS-102: Programming Fundamentals (PF)

**Assistant Professor, Computer Engineering
Department**

HITEC University Taxila Cantt, Pakistan

1. Memory and References

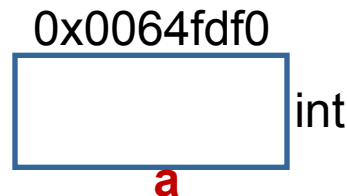
- Computer memory is a collection of different consecutive memory locations. These memory locations are numbered sequentially. Each variable is created at a unique memory location known as its **address**.
- A program may declare many variables to accomplish different tasks. A variable declaration reserves a set amount of space in memory for a particular object.
- When a variable is declared, the computer reserves an amount of space for that variable. The variable name is used to refer to that memory space.

1. Memory and References (continued...)

- A variable declaration associates the following three attributes to the variable:
 - *Variable name*
 - *Variable type*
 - *Variable memory address*
- The following statement declares an integer variable:
 - `int a;` // the name of variable is **a** and the type of variable is **int**.
- The memory address of the variable is unknown. Computer creates the variable at any available location in the memory.

1. Memory and References (continued...)

- The memory address is a *hexadecimal number* that refers to a particular location in the memory as follows:



- The above box represents the memory location allocated for a variable `a`. The hexadecimal value above the box is its assumed address. The variable `a` will occupy 2 bytes in memory.
- The actual address of the variable `a` can be displayed by using *reference operator* `&`. It is also known as *address operator*. The reference operator is used to access the memory address of a variable. The following statement will display the address of `a`: `cout<<&a;`

1. Memory and References (continued...)

Program 1.1

```
#include<iostream>
using namespace std;
int main()
{
    int n = 10;
    cout<<" The value of n:"<<n<<endl;
    cout<<" The memory address of n:"<<&n<<endl;
    return 0;
}
```

Output

```
The value of n:10
The memory address of n:0x6ffe0c
-----
```

2. Pointers

A **pointer** is a **variable** that is used *to store a memory address*. The reference operator is used to access the memory address of a variable and store it in a pointer.

2.1 Pointer Declaration

The method of declaring a pointer is same as declaring a simple variable. An asterisk ' * ' is used in the declaration that indicates that the variable is a pointer variable.

Syntax

The syntax of declaring a pointer is as follows:

DataType *var;

DataType: It is the type of variable pointed by the pointer variable.

***** : It indicates that the variable is a pointer variable.

var: It is the name of the pointer variable.

2. Pointers (continued...)

Example:

- `int *p;` // It declares a pointer variable *p*. The data type *int* indicates that it can store the memory address of an integer variable. The data type of a pointer must be same as the data type of the variable whose memory address is to be stored in the pointer.
- It is also possible to declare many pointer variables in one statement as follows: `float *p1, *p2;`.

2. Pointers (continued...)

Program 2.1:

```
#include<iostream>
using namespace std;
int main()
{
    int n;
    int *ptr;
    cout<<" Enter an integer: ";
    cin>>n;
    ptr = &n;
    cout<<" The value of n: "<<n<<endl;
    cout<<" The address of n: "<<ptr<<endl;
    return 0;
}
```

Output

```
Enter an integer: 25
The value of n: 25
The address of n: 0x6ffe04
```

2. Pointers (continued...)

How Program 2.1 Works?

It declares an integer variable *n* and an integer pointer *ptr*. It inputs a value from the user and stores it in *n*. The address of *n* is assigned to *ptr* using address of operator. Finally, the value and address of the variable is displayed.

2.2 The 'void' Pointer

The type of pointer variable and the type of variable it refers must be same. It restricts the use of a pointer variable to specific type of variables. A pointer variable can store the memory address of any type of variable if it is declared as *void*. The keyword '*void*' is used as the data type of the pointer as follows: `void *p;` // this statement declares a pointer variable *p*. The data type of the pointer is *void*. It means that it can store the memory address of any type of variable.

2. Pointers (continued...)

Program 2.2:

```
#include<iostream>
using namespace std;
int main()
{
    int n = 10;
    float f = 25.18;
    char c = '$';
    void *ptr;
    ptr = &n;
    cout<<" The value of n: "<<n<<endl;
    cout<<" The address of n:"<<ptr<<endl;
    ptr = &f;
    cout<<" The value of f: "<<f<<endl;
    cout<<" The address of f:"<<ptr<<endl;
    ptr = &c;
    cout<<" The value of c: "<<c<<endl;
    cout<<" The address of c:"<<ptr<<endl;
    return 0; }
```

Output

```
The value of n: 10
The address of n:0x6ffe04
The value of f: 25.18
The address of f:0x6ffe00
The value of c: $
The address of c:0x6ffdff
```

2. Pointers (continued...)

How Program 2.2 Works?

It declares and initializes these variables *int*, *float* and *char*. It also declares a *void* pointer variable. The pointer can refer to different types of variables. The program stores the memory addresses of the variables *n*, *f* and *c* one by one and displays the values and memory addresses of these variables.

2.3 Dereference Operator

The *dereference operator* is used to access the value of the variable whose memory address is stored in pointer. It is denoted by ‘***’. It is also called *indirection operator*. It can also be used to input a value in the variable and process the data stored in the variable.

2. Pointers (continued...)

Program 2.3:

```
#include<iostream>
using namespace std;
int main()
{
    int a, b, s, *p1, *p2;
    p1 = &a;
    p2 = &b;
    cout<<" Enter an integer:";
    cin>>*p1;
    cout<<" Enter an integer:";
    cin>>*p2;
    s = *p1 + *p2;
    cout<<*p1<<" + "<<*p2<<" = "<<s<<endl;
    return 0;
}
```

Output

```
Enter an integer:13
Enter an integer:6
13 + 6 = 19
```

2. Pointers (continued...)

How Program 2.3 Works?

It declares three integer variables and two pointers. The pointer ***p1*** refers to the memory address of ***a*** and ***p2*** refers to ***b***. The program uses indirection operator ‘*******’ with pointers to access the memory addresses of integer variables. It inputs integer values, adds the values and displays the result.

2.4 Pointer Initialization

The process of assigning a memory address to a pointer at the time of declaration is called ***pointer initialization***. C++ does not initialize variables automatically. Therefore, *a pointer variable should be initialized so that it may not point to anything invalid.* The pointer can be initialized to any valid memory address. It can also be initialized to a NULL or 0 value.

2. Pointers (continued...)

Syntax

The syntax to initialize pointers is as follows:

DataType *P = &Variable;

DataType: It is the type of variable pointed by the pointer variable

***P :** It indicates that the pointer variable to be initialized.

& : It is the address operator that is used to access memory address of a variable.

Variable: It is the name of variable whose memory address is assigned to the pointer.

Example

int n = 100;

int *p1 = &n;

int *p2 = NULL;

The pointer **p1** is initialized to the memory address of variable **n**. The pointer **p2** is initialized to **NULL**.

2. Pointers (continued...)

Program 2.4:

```
#include<iostream>
using namespace std;
int main()
{
    int a;
    int *ptr = &a;
    cout<<" Enter an integer: ";
    cin>>*ptr;
    cout<<" You entered: "<<*ptr;
    return 0;
}
```

Output

```
Enter an integer: 100
You entered: 100
-----
```

3. Operations on Pointers

The arithmetic operations on pointers work differently than normal integer data types. *Only **addition** and **subtraction** operations can be performed on pointers.* The effect of both addition and subtraction depends on the size of data type of the pointer.

3.1 Pointer Addition

The addition operation on pointer is used to move the pointer reference forward in the memory. The change of memory address referenced by the pointer depends on the data type of pointer. If the increment operator is used with an *integer pointer*, it will change the reference by **2 Bytes**. If the increment operator is used with a *character pointer*, it will change the reference by **1 Byte**. If the increment operator is used with a *long pointer*, it will change the reference by **4 Bytes**.

3. Operations on Pointers (continued...)

Example

Suppose there are three pointers as follows:

```
char *pChar;
```

```
short *pShort;
```

```
long *pLong;
```

Suppose the above pointers are pointing to the memory locations 1000, 2000 and 3000 respectively. The use of increment operator on the pointer will change the reference of the pointers. The result of increment operator on each of the pointers is shown in figure 1 next slide.

3. Operations on Pointers (continued...)

Example

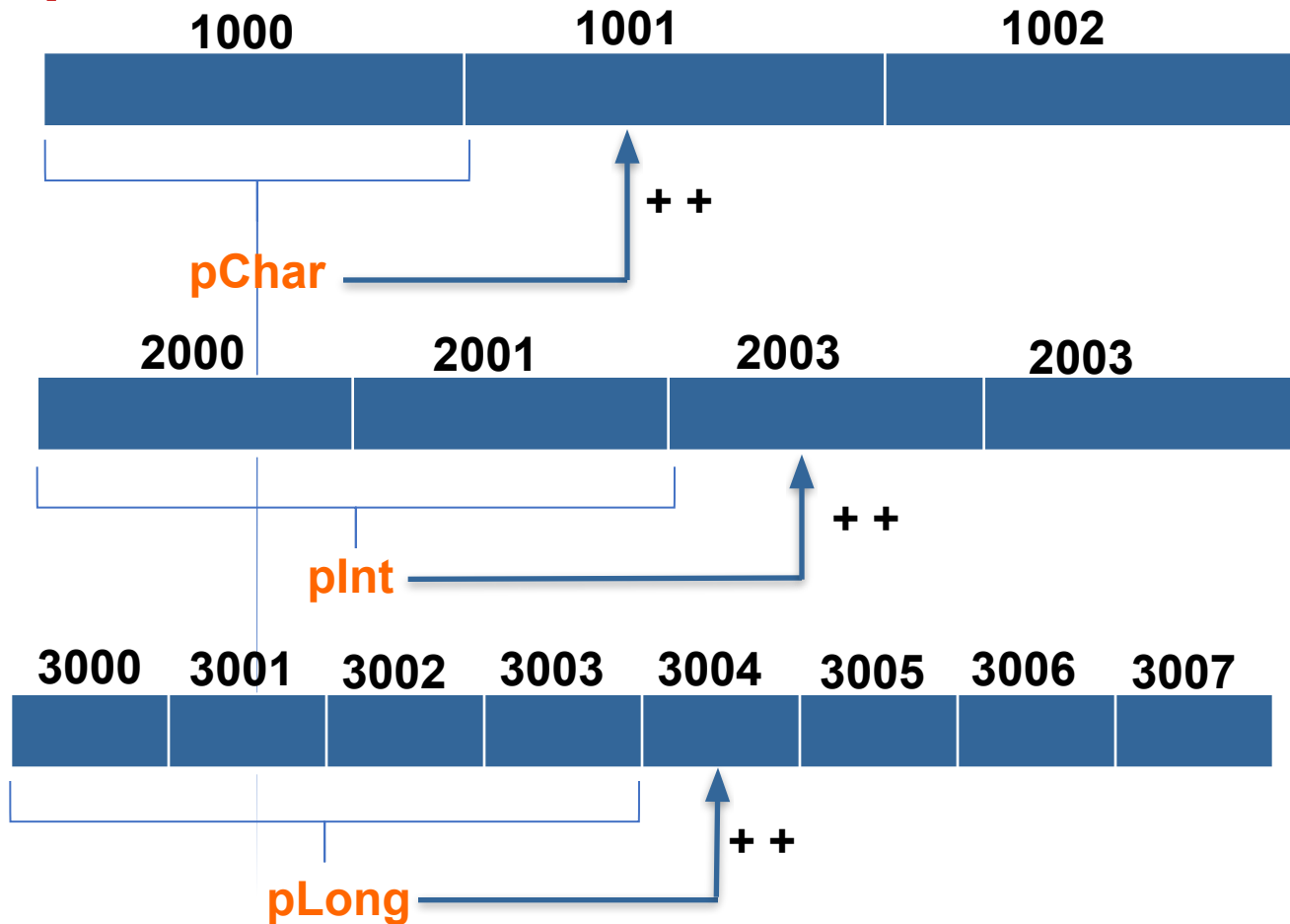


Figure 1

3. Operations on Pointers (continued...)

Figure 1 (previous slide) shows that the increment operator works differently on each pointer. The type of **pChar** pointer is **char**. The increment operator moves the reference of **pChar** by **1 byte**. The type of **plnt** pointer is **int**. The increment operator moves the reference of **plnt** by **2 bytes**. The type of **pLong** is **long**. The increment operator moves the reference of **pLong** by **4 bytes**.

3.2 Pointer Subtraction

The subtraction operation on pointer is used to move the pointer reference backward in the memory. The change of memory address referenced by the pointer depends on the data type of pointer. If the decrement operator is used with an integer, it will change the reference by 2 bytes. If the decrement operator is used with a character pointer, it will change the reference by 1 byte.

4. Pointers and Arrays

The pointers can also be used with arrays. An array is a collection of many elements of same type. All elements of the array are stored at consecutive memory locations. A pointer can access all elements of an array if the address of first element is assigned to it. The name of array represents the address of its first element. The address of first element can be assigned to a pointer by assigning the name of the array to pointer. The pointer then can access the remaining elements as they are stored consecutively in the memory.

Example:

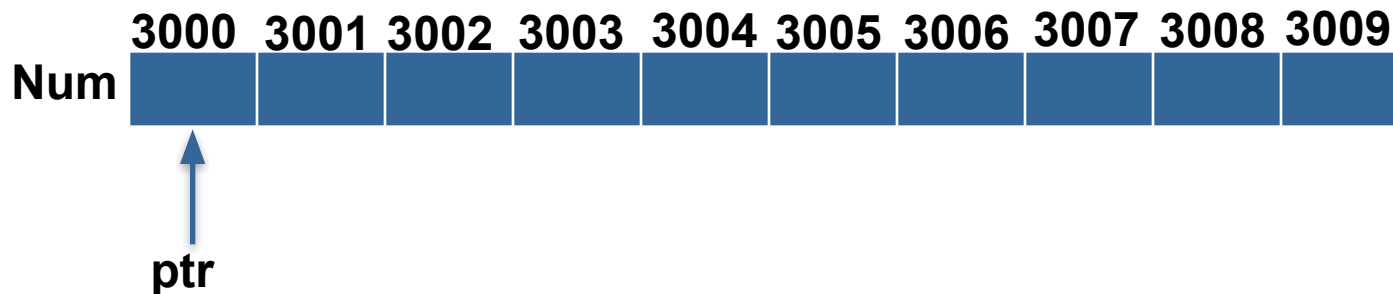
```
int Num[10];    //statement 1
```

```
int *ptr;       //statement 2
```

```
ptr = Num;      //statement 3
```

4. Pointers and Arrays (continued...)

The first statement of an example in previous slide declares an array of integers **Num** with 10 elements. The second statement declares a pointer variable **ptr** of type integer. The last statement assigns the address of first element of array **Num** to pointer **ptr** as follows:



4. Pointers and Arrays (continued...)

4.1 Accessing Array Elements with Pointers

The array elements can be accessed with pointers by moving the pointer to the desired element. The contents of an array elements can be accessed using dereference operator ‘ * ’. The pointer reference can be moved forward and backward by using increment operator ++ and decrement operator --.

Example:

```
int Num[5] = {10, 20, 30, 40, 50}; //statement 1
```

```
int *ptr = Num; //statement 2
```

```
cout<<*ptr; //statement 3
```

```
ptr++; //statement 4
```

```
cout<<*ptr; //statement 5
```

4. Pointers and Arrays (continued...)

The first statement of an example in previous slide declares an array *Num* and initializes it with five values. The second statement declares a pointer *ptr* and initializes it to the first element of the array *Num*. The third statement displays the value of first element that is 10. The fourth statement increments the pointer reference of the array. The pointer now refers to the second element. The last statement then displays the value of second element that is 20.

4. Pointers and Arrays (continued...)

The array elements can also be accessed without moving the pointer reference permanently as follows:

```
cout<<*ptr;    //statement 1
```

```
cout<<*(ptr+1); //statement 2
```

```
cout<<*(ptr+2); //statement 3
```

The first statement displays the value of first element as the pointer refers to the first element of the array i.e., element at index 0. The second statement displays the value of second element. Here **'1'** represents one byte. One byte is added to the first element of the array that results in the second element of the array i.e., element at index 1. The third statement displays the value of third element. Here **'2'** represents two bytes. Two bytes are added to the first element of the array that results in the third element of the array i.e., element at index 2.

4. Pointers and Arrays (continued...)

Program 2.5: Write a program to input five integers in an array and display them using a pointer.

Output

```
#include<iostream>
using namespace std;
int main()
{
    int marks[5], *ptr;
    cout<<"Enter 5 marks: "<<endl;
    for (int i=0;i<5;i++)
    {
        cin>>marks[i];
    }
    ptr = marks;
    cout<<"You have entered the following values: ";
    for(int i =0;i<5;i++)
    {
        cout<<*ptr++<<"\t";
    }
    return 0; }
```

```
Enter 5 marks:
10
20
30
40
50
You have entered the following values: 10      20      30      40      50
-----
Process exited after 8.915 seconds with return value 0
Press any key to continue . . .
```

4. Pointers and Arrays (continued...)

Program 2.6: Write a program that inputs five floating-point values in an array and displays the values in reverse order.

```
#include<iostream>
using namespace std;
int main()
{
    float arr[5], *ptr;
    cout<<"Enter five floating-point values: "<<endl;
    for (int i=0;i<5;i++)
    {
        cin>>arr[i];
    }
    ptr = &arr[4];
    cout<<"The values in reverse order: "<<endl;
    for(int i =0;i<5;i++)
    {
        cout<<*ptr--<<endl;
    }
    return 0;}
```

Output

```
Enter five floating-point values:
10
20
30
40
50
The values in reverse order:
50
40
30
20
10
```

5. Array of Pointers

An array of pointers is an array in which each element is a pointer. Each element in the array can store a memory address. The array can store the memory addresses of different objects of same type.

Program 2.7:

```
#include<iostream>
using namespace std;
int main()
{
    int *ptr[3],a,b,c;
    ptr[0] = &a;
    ptr[1] = &b;
    ptr[2] = &c;
    cout<<"Enter three integers: "<<endl;
    cin>>a>>b>>c;
    cout<<"You entered the following values: "<<endl;
    for (int i=0;i<3;i++)
    {cout<<*ptr[i]<<endl;}
    return 0; }
```

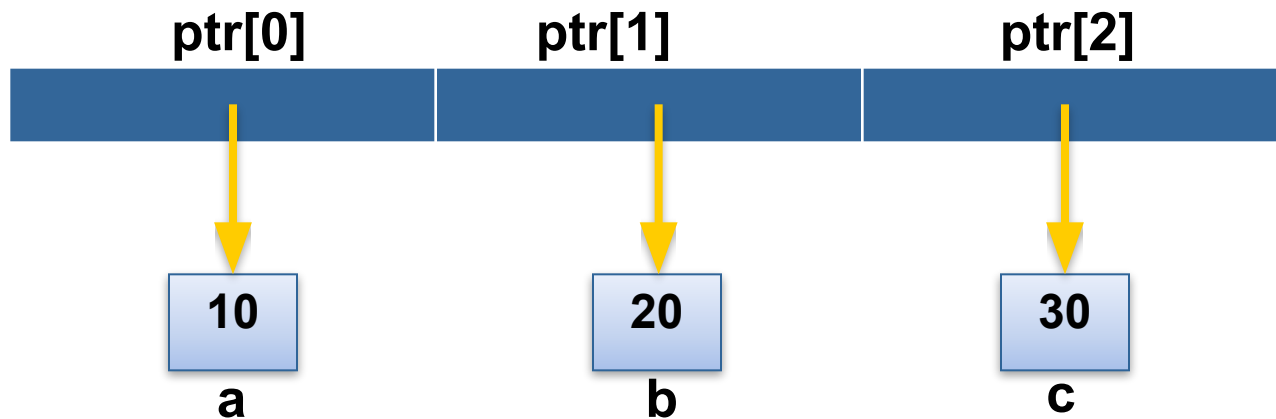
Output

```
Enter three integers:
10
20
30
You entered the following values:
10
20
30
```

5. Array of Pointers (continued...)

How Program 2.7 Works?

The program declares an array of (integer) pointers and three integer variables. Each element of pointer array refers to different variable. The program inputs the values in the variables and displays them using array of pointers.



6. Pointers and Functions

A parameter can be passed to function using pointers. The address of *actual parameter* is passed to the *formal parameter* if the formal parameters are defined as pointers.

6. Pointers and Functions (continued...)

Program 2.8: Write a program that inputs two integers and passes them to a function using pointers. The function exchanges the values and the program finally displays the values.

```
#include<iostream>
using namespace std;
void exchange(int*m, int*n);
int main()
{
    int n1,n2;
    cout<<"Enter two integers: ";
    cin>>n1>>n2;
    cout<<"Values before swapping: "<<endl;
    cout<<"n1 = "<<n1<<endl;
    cout<<"n2 = "<<n2<<endl;
    exchange(&n1, &n2);
    cout<<"Values after swapping: "<<endl;
    cout<<"n1 = "<<n1<<endl;
    cout<<"n2 = "<<n2<<endl;
    return 0;}
1
```

```
void exchange(int*m, int*n)
{
    int temp;
    temp = *m;
    *m = *n;
    *n = temp;
}
2
```

Output

```
Enter two integers: 10 20
Values before swapping:
n1 = 10
n2 = 20
Values after swapping:
n1 = 20
n2 = 10
```

6. Pointers and Functions (continued...)

Home Work

Program 2.9: *Write a program that declares an integer in main() function. It also declares two functions. One function inputs a value in the variable defined in the main() function. The second function doubles the value of that variable.*

7. Memory Management with Pointers

The process of allocating and deallocating memory is known as **memory management**. The operating system sets up different areas of memory according to the requirement when a program starts execution. Some important areas in the memory are as follows:

- **Global name space:** It is used to store global variables.
- **Registers:** These are special memory units built into CPU.
- **Code space:** It contains the program code.
- **Stack:** It contains local variables and function parameters. It is also known as **static memory**.
- **Free store:** It is the remaining memory. It is also known as **heap** or **dynamic memory**. It is used for dynamically allocating memory blocks during program execution.

7. Memory Management with Pointers (continued...)

7.1 *Dynamic Variables*

A variable that is created during program execution is called **dynamic variable**. The dynamic variables can be created in C++ with the help of pointers. C++ provides two operators **new** and **delete** operators for dynamic variables. The **new** operator is used to create dynamic variables and **delete** operator is used to delete dynamic variables during program execution.

7.2 *The new Operator*

The **new** operator is used to allocate memory dynamically. It is followed by the type of object for which the memory is to be allocated. The compiler allocates the amount of memory according to the type of object. The **new** operator returns a memory address. The returned address must be assigned to a pointer.

7. Memory Management with Pointers (continued...)

The pointer is then used to access the memory location and process the values stored in that address. The **new** operator can be used to create simple variable, an object or an array of objects.

Syntax

The following syntax is used to create one variable:

new Datatype;

The following syntax is used to create an array dynamically:

new Datatype[length];

7. Memory Management with Pointers (continued...)

Example

```
int *ptr; //statement 1
```

```
ptr = new int; //statement 2
```

The first statement declares an integer pointer variable. The second statement uses the *new* operator to create an integer variable in the memory dynamically. The address of the allocated memory is stored in the pointer *ptr*.



The above figure shows that the newly created memory has no name. It can only be accessed using the pointer.

7. Memory Management with Pointers (continued...)

Example

```
int *ptr; //statement 1
```

```
ptr = new int[5]; //statement 2
```

The first statement declares an integer pointer variable. The second statement uses the *new* operator to create an array of integers (of length 5) in the memory dynamically. The address of first element of the array is stored in the pointer *ptr*.



7. Memory Management with Pointers (continued...)

7.3 The delete Operator

The delete operator deallocates the memory and returns the allocated memory back to the **free store**. It takes a pointer variable as parameter and releases the memory referred by the pointer. The dynamically created object should be deleted when it is no more required. The pointer declared in a function is treated as a local variable.

Syntax

The following syntax is used to delete one variable dynamically:

delete variable;

The following syntax is used to delete an array dynamically:

delete [] variable;

7. Memory Management with Pointers (continued...)

Example

```
delete ptr;
```

The above statement will deallocate the memory referred by the pointer *ptr*. The use of delete operator is slightly different when it refers to an array. The following statement is used if the pointer *ptr* refers to an array:

```
delete[ ] ptr;
```

7. Memory Management with Pointers (continued...)

Program 2.10: Write a program that uses new operator to create an integer, inputs value in it and then displays that value.

Output

```
#include<iostream>
using namespace std;
int main()
{
    int *ptr;
    ptr = new int;
    cout<<" Enter an integer: ";
    cin>>*ptr;
    cout<< "You entered " <<*ptr<<endl;
    cout<< "It is stored at " <<&ptr<<endl;
    delete ptr;
    return 0;
}
```

```
Enter an integer: 50
You entered 50
It is stored at 0x6ffe08
-----
```

7. Memory Management with Pointers (continued...)

Home Work

Program 2.11: *Write a program that inputs length from the user and declares an array of integers of the length specified by the user. The program then inputs values in array and displays them.*

References

1. *“C++ Programming: From Problem Analysis to Program Design”, 6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *“Object Oriented Programming in C++,” 4/ed by Robert Lafore (2001) .*
3. *“C++ How to Program,” 5/ed by Deitel & Deitel (2005).*
4. *“Program with C++ Object Oriented Programming Aikman Series”, by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*